

is generated by the Web application/Web server, and does not require the browser to pass authentication tokens along with the request.

- Form-based session tokens, in which hyperlinks are replaced with HTML forms that contain session identifiers in hidden form fields.

Though explicit authentication is immune to CSRF, it has other problems, which we will discuss in a while.

The main reason that CSRF works is that Web applications using implicit authentication mechanisms do not verify that a state-changing request was created within the Web application. Because of the implicit authentication of the user, the attacker can easily control the user's session.

Another underlying factor that enables CSRF attacks is the application's use of predictable URL/form actions in a repeatable way.

Some myths about CSRF

Myth: CSRF is just a special case of XSS.

Fact: XSRF is a separate vulnerability from XSS, with a different solution. XSS protection won't stop XSRF attacks, though it is also important to guard against XSS on a priority basis.

Myth: Applications aren't vulnerable to CSRF if they use multi page forms to perform actions.

Fact: While multi page forms certainly make exploitation harder, attackers usually can exploit them. Attackers frequently use multiple *IFRAMES* when building multi page form CSRF exploits.

Myth: CSRF is solvable by forcing all sensitive requests over *POST* while denying *GETs*.

Fact: While *POSTs* can be more difficult to exploit, they certainly are exploitable, as shown in Scenario 1. Using *POST* rather than *GET* can provide a defence only against local CSRF attacks; *POST* requests can still be created with hidden *IFRAMES* on foreign Web pages. Forms can mislead users about what they are sending, and where, and scripting can lead to automatic submission. JavaScript is fully capable of sending *POST* requests via form submissions.

Myth: CSRF can be prevented by filtering based on the referrer header.

Fact: This option is very unreliable, since attackers can easily block the sending of the referrer header, through the use of certain browser and Flash exploits. Some browsers also omit the referrer header when they are being used over SSL. Moreover, many firewalls and anti-spyware software often drop referrers, in their default mode, without letting users know. But, using referrers can be viewed as another incremental roadblock.

Myth: Browser enforcement of Same Origin Policy (SOP) prevents CSRF.

Fact: Browsers that implement SOP will prevent scripts from accessing the DOM of a page originating from another domain, or accessing cookies that originate from other domains. However, they do not prevent scripts from sending requests to other domains. Furthermore, when a script sends

a request to another domain (or when an *IMG* tag's *SRC* attribute is set to another domain), the browser will execute the request, and will also send any cookies it has that are valid for the domain, along with the request. (See box on SOP.)

Advanced uses of CSRF

The following advanced techniques that use CSRF attacks have been observed in recent years.

1. **Bypassing CSRF protections with click-jacking:** This recently-evolved technique can be used to bypass CSRF protection and submit *POST* method-based forms with attacker controlled data, using click-jacking. See the box on click-jacking for more information.
The best example of this attack is exploiting e-mail update services. Such services are quite common in Web applications. In this, the attacker manages to force victims to update their e-mail IDs with that of the attacker, so that the attacker can then compromise the victims' account by performing a password reset. This attack can occur even if the Web application contains tokens for CSRF protection. A cool and elaborate description of this is available at <http://blog.andlabs.org/2010/03/bypassing-csrf-protections-with.html>. Do check it out for more information.
2. **Using XSS to bypass CSRF protection:** This technique applies to websites that have an application that guards against CSRF, yet have pages that are vulnerable to XSS attacks. It's a big misconception that guarding against CSRF will also secure the application against XSS. Using XSS, attackers can bypass the CSRF protection, and can automate any action that can be done on the application, without problems. Attackers simply read the site-generated token from the response, and include that token with a forged request. One such example of this is attackers exploiting an XSS vulnerability of a website to add a fake user with administrator privileges, when the site is secured against CSRF attacks. A good discussion on this type of attack (with code) is available in the *Further Reading* section. Please do spend some time on it. It was by using an XSS vulnerability that the Samy worm bypassed MySpace's CSRF protection in 2005, infecting over one million accounts within just 20 hours of its release.
3. **Attacking intranets:** Exploiting intranets involves CSRF attacks through IP-based implicit authentication schemes. Most intranet Web servers are vulnerable to this type of attack. The fact that many intranet servers continue to use default passwords, leave hosts unpatched, and blindly rely on perimeter firewalls to block external attacks has given rise to intranet vulnerabilities to CSRF and malicious JavaScript. This malicious JavaScript, once behind the firewall, can attack the intranet; since it is independent of the operating system and the Web browser, it is hard to defend against.
One such example is port scanning of the intranet Web server, using JavaScript and CSRF. The JavaScript constructs a local (intranet) URL that contains the IP address and the port